

26-Arch Lab+ 实验报告

潘孝圆 24300240128

2026-06-19

1 完成内容

本次 Lab+ 在已有 Lab1-Lab6 实现基础上继续补充 bonus。主要完成内容如下：

- 保留并验证 Lab1 extra 的乘除法扩展支持。
- 保留 Nexys4 上板适配，继续使用非 Basys3 开发板配置。
- 保留 Lab5 的 Sv39 MMU、2 MiB/1 GiB hugepage 支持、特权级和 Lab6 异常中断实现。
- 新增 Lab+ atomic extension：实现 AMO W 系列指令以及 LR.W/SC.W，并接入 difftest atomic event。
- 新增前端性能优化：顺序取指提前发起请求、CBusArbiter idle fast path，以及 IBusToCBus 8B 指令行缓冲，减少连续取指空泡。
- 新增 PMP/privfull 支持：支持 pmpcfg0 中 8 个 PMP entry 的 OFF/TOR/NA4/NAPOT 匹配，产生 instruction/load/store access fault，并通过 lab+/4 privileged sys-test。
- 新增 EBREAK 断点异常支持：SYSTEM/funct12=0x001 触发同步异常 cause 3。
- 新增 FENCE/FENCE.I 合法 no-op 支持，提升编译器生成程序的兼容性。
- 新增 xv6 主线部分进展：补充真实 S-mode、SRET、异常/中断委托和 S 态 trap CSR 写入路径。
- 新增 S 态中断 pending 委托转换：当 mideleg 委托 SSIP/STIP/SEIP 时，将 swint/trint/exint 映射到对应 S pending 位。
- 新增 MMU page fault 与 PTE 权限检查：识别 Sv39 非 canonical 地址、无效 PTE、叶子页权限不满足、巨页 PPN 未对齐，并产生 instruction/load/store page fault。
- 新增 SUM/MXR 支持：MXR 允许 load 读取 execute-only 页，SUM 允许 S 态数据访问 U 页，同时保持 S 态不能从 U 页取指。
- 新增 PTE A/D 位硬件更新：叶子 PTE 的 A=0 或写访问 D=0 时，MMU 先写回更新后的 PTE，再继续最终访存。
- 新增仿真侧简化 virtio/disk MMIO：在 0x10001000 暴露 virtio-mmio 识别寄存器，并提供同步 512B sector 读写扩展。
- 新增 MMU page fault 定向单元测试，覆盖 instruction/load/store fault 和正常 load 翻译路径。
- 新增 S 态中断定向测试，覆盖 delegated STIP 从硬件 trint 进入 S trap 的路径。
- 新增 test-labplus-2/3/4、test-labplus-pagefault、test-labplus-sinterrupt 与 test-labplus-virtio Makefile 测试入口，并补入官方 Lab+ ready-to-run 测试文件。

本次新增通过的核心测试为 atomic extension、privileged/PMP sys-test、MMU page fault 定向测试、S 态中断定向测试和 simple virtio block MMIO 定向测试。lab+/4 全量 TEST=all 已完成 benchmark 和 sys-test，最终输出 Privileged test finished. Exit with code = 0。当前官方 all-test-privfull.bin 中未包含真实 ebreak 指令，breakpoint [X] 来自测试程序自身的占位输出；补充 EBREAK 后该输出仍不会变化，不影响最终 privileged 测试收尾。

2 Atomic Extension 设计

2.1 指令解码

新增 opcode 0101111 的 A 扩展解码，当前支持 funct3=010 的 32 位原子指令：AMOADD.W、AMOSWAP.W、AMOXOR.W、AMOAND.W、AMOOR.W、AMOMIN.W、AMOMAX.W、AMOMINU.W、AMOMAXU.W、LR.W、SC.W。

LR.W 要求 rs2=0。所有 AMO W 指令按照 store/AMO 地址对齐规则检查地址低两位，不对齐时产生 store/AMO address misaligned 异常。

2.2 两阶段访存

当前 CPU 是顺序单发结构，同一时刻只有一条指令处于访存阶段。普通 AMO 指令被实现为两阶段状态机：

```
read old word from memory
compute new word
write new word back to same address
commit rd = sign_extend(old word)
```

第一阶段读响应到达后，指令仍保持 mem_pending=1，并将同一个请求槽切换为写请求。这样不会让取指或下一条指令插入 AMO 的读写之间，满足单核测试中的原子性要求。

LR.W 只执行读请求，并记录 {addr[63:2], 2'b00} 作为 reservation 地址。SC.W 在 reservation 命中时发出写请求并返回 0；未命中时不访问内存，直接返回 1，并清除 reservation。

2.3 Difttest 事件

AMO 不能直接作为普通 store event 上报。修复方式如下：

- 普通 DifttestStoreEvent 屏蔽 dt_is_atomic。
- AMO 访存提交时额外上报 DifttestAtomicEvent。
- AMO load event 的 fuType 设置为 0x0f，使 difftest 按 atomic 路径处理。
- atomicData 使用未移位的 rs2，atomicOut 使用旧值，atomicMask 根据地址低三位生成。

3 Privfull 与 PMP 支持

3.1 jalr 不对齐异常

lab+/4 的 instr_misalign 使用 jalr 跳转到 target + 1。原实现使用清除 bit0 后的 jalr 目标判断是否不对齐，因此该项不会触发异常。为匹配课程测试，本次改为正常跳转目标仍使用

{raw_target[63:1], 1'b0}, 但 instruction address misaligned 判断和 mtval 使用未清 bit0 的 raw_target。该修改后 Lab6 的 instr_misalign 回归仍通过。

3.2 PMP 匹配与权限

实现了 PMP 多 entry 匹配, 用于覆盖 Lab+ privfull 测试并补齐更通用的 PMP 行为:

- 支持 pmpaddr0 到 pmpaddr7, pmpcfg0 中每个 8-bit 配置项对应一个 PMP entry。
- 支持每个 entry 的 OFF、TOR、NA4、NAPOT 地址匹配。
- TOR entry 使用前一个 pmpaddr 作为下界, entry0 的下界为 0。
- 低编号 entry 优先, 命中后使用该 entry 的 R/W/X/L 权限判断。
- 所有 PMP entry 都为 OFF 时不影响原有 Lab5/Lab6。
- PMP 激活后, U/S 态未命中 PMP 或权限不足会触发 access fault。
- M 态在命中 entry 未 lock 时绕过权限检查, 命中 lock entry 时按权限检查。

测试程序设置的 PMP 范围为 0x80006000 到 0x80007000, 即 4096B 用户区。PMP 激活后, 用户态访问该范围外的数据或跳转到该范围外取指都会触发异常。

3.3 异常接入

取指侧新增 if_id_access_fault。当 PMP 拒绝取指时, CPU 不再真的向内存发起请求, 而是向 decode 注入一条携带 fault 信息的占位指令, 随后走统一 trap 路径, 产生 cause 1。

数据侧在发起 dreq 之前检查 PMP:load/LR/AMO read 权限不足产生 cause 5;store/SC/AMO write 权限不足产生 cause 7; 地址不对齐优先级仍高于 access fault。

另外新增 fetch_priv 解决 trap/mret 重定向同周期的权限判断问题: trap 重定向到 mtvec 时按 M 态检查取指, mret 重定向时按返回后的权限检查取指。

3.4 EBREAK 断点异常

Lab+ privfull 输出中存在 Test breakpoint [X]。反汇编和二进制字节检查显示当前官方 all-test-privfull 没有 ebreak 编码 0x00100073, 该项不会真正触发断点异常。为补齐异常类型, 本次仍在 CPU 中新增 EBREAK:

- 在 SYSTEM 指令 funct3=000 且 instr[31:20]=12'h001 时识别为 EBREAK。
- EBREAK 作为合法指令进入同步异常路径。
- trap cause 设置为 3, mtval 保持 0。
- 已重跑 make test-labplus-3、Lab+ privfull sys-test 和 Lab6 sys-test, 均保持通过。

3.5 FENCE/FENCE.I 兼容

当前 CPU 没有 cache, 也没有乱序访存或 speculative 执行, 因此普通 FENCE 和 FENCE.I 不需要真实刷新硬件状态。本次将 opcode 0001111 中 funct3=000/001 的指令作为合法 no-op 提交:

- FENCE 用于兼容内存顺序屏障。
- FENCE.I 用于兼容指令流同步屏障。
- 两者不写寄存器、不发访存请求、不触发流水线重定向。
- 已随 atomic、Lab+ privfull 和 Lab6 回归一起验证。

3.6 xv6 主线部分进展：S-mode 与委托

官方 Lab+ 主 Track 是尝试运行 xv6。当前仓库没有 xv6 镜像或源码，因此本次先补 xv6 boot 的 CPU 前置能力，而不是直接实现磁盘 MMIO。新增内容如下：

- 新增 `PRIV_S=2'b01`，`mret` 可以返回到 S 态，`diffest` 的 privilege mode 也能看到 S 编码。
- 新增 SRET 解码，S/M 态可执行，返回到 `sstatus.spp` 指定的 U/S 态，并按规范更新 `sstatus.sie/spie/spp`；
- `ECALL` 在 S 态触发 cause 9，U 态仍为 cause 8，M 态仍为 cause 11。
- 开放 `mideleg` 常见可委托异常位，开放 `mideleg` 的 SSIP/STIP/SEIP 位。
- `trap` 被委托到 S 态时，跳转 `stvec`，写入 `sepc/scause/stval`，并更新 `sstatus.sie/spie/spp`；未委托时保持原 M 态 `trap` 路径。
- 增加 S 级中断 evaluate 框架：当 `mip/sip`、`mie/sie`、`mideleg` 同时打开时，U/S 态可进入 S `trap`。
- 增加简化的 CLINT/PLIC 到 S pending 转换：当 `mideleg.SSIP/STIP/SEIP` 对应位打开时，将外部输入 `swint/trint/exint` 同步镜像到 `mip.SSIP/STIP/SEIP`，使 S 态只打开 `sie` 对应位即可接收 supervisor software/timer/external interrupt。

这一部分还不能直接跑完整 xv6，因为仍缺少 `virtio`/磁盘 MMIO 或替代块设备模型。但它把 xv6 所需的 Supervisor `trap` 基础路径补上了，并通过现有 Lab5、Lab6 与 Lab+ `privfull` 回归确认没有破坏原 U/M 行为。

3.7 MMU page fault 与 PTE 权限检查

原 MMU 在页表项无效时会把最终物理地址置 0 并继续发起访存，这对 Lab5 的正常路径足够，但不适合继续做 xv6/异常测试。本次在 CBus 响应中增加 `page_fault` 标记，并把该标记一路传回取指和数据访存路径：

- CBus 请求增加 `is_instr`，MMU 能区分取指、load 读和 store/AMO 写。
- CBus/IBus/DBus 响应增加 `page_fault`，MMU 发现翻译失败时返回一拍 `fault` 响应，不再访问物理地址 0。
- 取指 `page fault` 进入 decode 统一异常路径，产生 cause 12，`mtval/stval` 写入 faulting virtual address。
- `load/LR/AMO read page fault` 在数据响应阶段产生 cause 13。
- `store/SC/AMO write page fault` 在数据响应阶段产生 cause 15，并阻止该访存提交到 `diffest`。
- MMU 检查 Sv39 canonical 地址、V=0、W=1 && R=0、第三级仍非叶子、叶子页 R/W/X/U 权限和 1 GiB/2 MiB 巨页 PPN 对齐。

数据侧 `page fault` 是响应期异常，不在 decode 阶段就能确定。因此本次额外补了 WB `trap` 重定向：当 `dresp.page_fault=1` 时清除等待中的访存、写入 `trap CSR`，并清空 IF/ID，防止后续指令越过 faulting load/store 提交。

3.8 PTE A/D 位硬件更新

RISC-V Sv39 的叶子 PTE 中，A 表示 accessed，D 表示 dirty。为了让不预先置 A/D 的页表也可以运行，本次在 MMU 页表 walker 中加入硬件更新路径：

- 叶子 PTE 权限检查通过后，如果 A=0，MMU 对该 PTE 地址写回 A=1。
- 对 `store/SC/AMO write`，如果 D=0，同一次写回同时置 D=1。

- instruction fetch 和 load 只需要置 A，不置 D。
- PTE 写回完成后，MMU 再发起原本的最终物理地址访问。
- 非法 PTE、权限不足、superpage PPN 未对齐等 fault 不会更新 A/D 位。

实现上新增 S_AD_UPDATE 状态，复用已有 CBus 对页表项地址发起 64 位写事务；外部 MMU 接口保持不变。

3.9 SUM/MXR 权限补充

在 MMU 中继续补充了 mstatus.sum 和 mstatus.mxr 对页表权限的影响：

- mstatus.mxr=1 时，load 可以读取 X=1/R=0 的 execute-only 页。
- mstatus.mxr=0 时，load 仍要求 R=1。
- U 态访问页表项时仍要求 PTE_U=1。
- S 态访问 supervisor 页时要求 PTE_U=0。
- S 态数据访问用户页时，只有 mstatus.sum=1 才允许。
- S 态取指仍然禁止从用户页执行，即使 SUM=1 也会触发 instruction page fault。

实现上，core 将当前 mstatus 输出到 MMU，MMU 在叶子 PTE 权限判断时组合使用 priv_mode、is_instr、is_write、SUM/MXR 和 R/W/X/U 位。该改动不改变 CSR 写入路径，MSTATUS_MASK/SSTATUS_MASK 之前已经开放了 SUM/MXR 位，因此软件可以通过 mstatus/sstatus 正常设置。

3.10 简化 virtio/disk MMIO

为了继续推进 xv6 所需的块设备方向，本次先在仿真侧加入一个简化块设备，而不是直接实现完整 virtio queue。实现方式是新增 SimMemoryWithVirtio 包装器：它拦截 0x10001000 到 0x100011ff 的 MMIO 访问，其它请求继续转发给原来的 RAMHelper2，因此 UART、mtime/mtimecmp、msip 和普通 RAM 行为保持不变。

兼容识别寄存器：

- 0x10001000：低 32 位为 virtio magic 0x74726976，高 32 位为 version 2。
- 0x10001008：低 32 位为 device id 2，表示 block device；高 32 位为 vendor 0x554d4551。

简化同步块设备扩展：

- 0x10001100：sector index。
- 0x10001108：DMA memory address。
- 0x10001110：command，1 表示从 disk 读 512B 到 RAM，2 表示从 RAM 写 512B 到 disk。
- 0x10001118：status，0 表示完成，1 表示未知命令，2 表示 sector 越界。
- 0x10001120：sector 数量，当前为 16。
- 0x10001128：sector 大小，当前为 512B。

该设备是同步完成模型，适合后续写一个很薄的软件适配层先绕开完整 virtio descriptor/avail/used ring；完整 xv6 原生 virtio 驱动仍需要后续实现 queue 和中断通知。

4 前端性能优化

4.1 顺序取指提前

优化前取指状态机只在以下条件满足时发起新请求：

```
!if_pending && !if_id_valid && !mem_pending
```

优化后新增 if_can_request:

```
!if_pending && !mem_pending
&& (!if_id_valid || (id_consume && !id_redirect))
```

当当前 if_id 指令在本周期被消费, 并且不是 trap、跳转、taken branch、CSR 或 MRET 时, CPU 同周期发起下一条顺序取指请求。对于重定向类指令, 仍然等待目标写入 pc 后再取指。

4.2 CBusArbiter fast path

原 CBusArbiter 在 idle 状态下不会直接把选中的请求透传到输出总线, 因此每个请求都会固定多等一拍。本次改为 idle 且存在请求时, oreq 直接等于当前选中的 selected_req; 如果下游同周期返回 last, 仲裁器不进入 busy; 如果请求未完成, 才锁存 index 并进入 busy。该优化对取指和数据访存都生效。

4.3 8B 指令行缓冲与 WB 旁路

现有 CBus/RAM 读响应本身是 64 位宽。原 IBusToCBus 每条 32 位指令都会重新经过 DBus/CBus/MMU 路径, 即使下一条指令就在同一个 8B 对齐块的另一半, 也会再消耗一次总线请求。本次在 IBusToCBus 中增加一个很小的 8B instruction line buffer:

- miss 时仍按原方式发起 4B instruction request, 保持对外请求语义不变。
- 响应到达后缓存 {addr[63:3], 3'b000} 对齐的 64 位数据。
- 下一次取指如果命中同一个 8B 行, 直接按 addr[2] 返回低/高 32 位, 不再访问 CBus。
- trap、跳转、CSR、FENCE/FENCE.I 和 SFENCE.VMA 等会改变取指流或地址翻译状态的指令通过 if_flush 失效该缓冲。

取指空泡减少后, 原本被慢取指掩盖的 load/use 时序风险会暴露出来: WB 本周期写回的寄存器可能被同周期 decode 读取。为此在 core 中补充 WB 到 decode 的简单旁路, 当 wb_valid && wb_wen && wb_rd == rs1/rs2 时优先使用 wb_data。这保证前端加速不会破坏已有 Lab4 这类密集 load/use 程序。

另外尝试过对 JAL/JALR/taken branch 目标同周期提前取指。该路径在功能上保留, 只对普通控制流重定向生效; trap/CSR/FENCE/SFENCE 仍走保守 flush。ready-to-run 的几个直线型测试主要受顺序取指和 8B 行缓冲影响, 分支目标提前取指收益较小。

4.4 周期对比

测试	原始周期	顺序提前	fast path	8B 缓冲	总减少	最终 IPC
atomicity.bin	372	317	249	196	47.3%	0.281
lab1-extra-test.bin	185976	153201	120425	93022	50.0%	0.352
lab4-test.bin	208529	177990	139050	110574	47.0%	0.296

MicroBench qsort 采样从 7 ms 降到 6 ms。lab+/4 TEST=all 中的 benchmark 结果如下:

CoreMark: 583 ms, 17 Iterations/Sec

Dhrystone: 813 ms, 21 Marks

STREAM Copy/Scale/Add/Triad: 19.2 / 1.1 / 2.3 / 1.1 MB/s

TEST=all 在 sys-test 输出 Privileged test finished. Exit with code = 0 后仍可能保持仿真进程不退出, 因此实际运行用 timeout 截断; 报告中的 benchmark 和 sys-test 结果均已出现在截断前。

5 代码修改

- `vsrc/src/core.sv`: 新增 AMO 解码、AMO 结果计算、reservation 记录; 扩展访存状态机; 新增 diffest atomic event; 新增 `if_can_request`; 新增普通 JAL/JALR/taken branch 目标提前取指、`if_flush` 输出和 WB 到 decode 的寄存器旁路; 新增 8-entry PMP 匹配、取指 access fault 注入、load/store access fault; 新增 `fetch_priv`; 新增 EBREAK 解码和 breakpoint exception cause 3; 新增 FENCE/FENCE.I 合法 no-op 解码; 新增 S-mode、SRET、medeleg/mideleg 委托、S 态 trap CSR 写入; 新增委托后的 SSIP/STIP/SEIP pending 镜像, 支持 swint/trint/exint 进入 S trap; 新增 instruction/load/store page fault 接入和数据访存 fault 后的 WB trap 清流水; 输出当前 `mstatus` 给 MMU 使用。
- `vsrc/SimTop.sv`: 仿真顶层改为实例化 `SimMemoryWithVirtio`, 在原 RAM/CLINT 行为前增加 `virtio/simple-block` MMIO 拦截; 显式连接 `if_flush`, 供取指侧 instruction line buffer 在控制流/地址翻译变化时失效。
- `vsrc/VTop.sv`、`vivado/src/with_delay/soc_top.sv`: 连接 `if_flush` 到 `IBusToCBus`, 保持仿真顶层和上板顶层一致。
- `vsrc/include/common.sv`: 扩展 CBus/IBus/DBus 响应, 增加 `page_fault`; 扩展 CBus 请求, 增加 `is_instr` 标记。
- `vsrc/util/MMU.sv`: 新增 fault 状态和 PTE 权限检查, 不再将无效 PTE 转成物理地址 0; 支持 Sv39 canonical 地址检查和巨页 PPN 对齐检查; 支持 SUM/MXR 对 S/U 态数据访问和 execute-only 页读取的影响; 新增 `S_AD_UPDATE` 状态, 支持硬件置 PTE A/D 位。
- `vsrc/util/IBusToCBus.sv`、`vsrc/util/DBusToCBus.sv`: 传递 `is_instr` 与 `page_fault`; `IBusToCBus` 新增 8B instruction line buffer, 命中同一 64 位取指行时直接返回另一条 32 位指令。
- `vsrc/mycpu_top.sv`、`vsrc/util/CBusToAXI.sv`、`vivado/src/with_delay/soc_top.sv`: 对真实内存响应源固定 `page_fault=0`。
- `vsrc/include/csr.sv`: 修正 `SSTATUS_MASK`, 开放 SIE/SPIE/SPP 等 S 态状态位; 设置 `MEDELEG_MASK/MIDELEG_MASK`, 支持常见异常和 S 级中断委托; 新增 `pmpaddr1` 到 `pmpaddr7` CSR 地址常量。
- `vsrc/util/CBusArbiter.sv`: idle 状态下直接透传当前请求, 同周期完成的请求不再进入 busy。
- `vsrc/test/mmu_page_fault_tb.sv`: 新增独立 Verilator testbench, 直接实例化 MMU; 构造固定三层 Sv39 页表, 验证 instruction/load/store page fault 与正常 load 翻译。
- `vsrc/test/s_interrupt_pending_tb.sv`: 新增独立 core 级 Verilator testbench, 用手写 CSR 指令初始化委托和 S 态中断使能; 验证 `trint` 在 `mideleg.STIP=1`、`mie.STIE=1`、`mstatus.SIE=1` 后进入 S trap, `scause` 为 interrupt cause 5。

- `vsrc/test/difftest_stubs.sv`: 为独立 core test 提供空 difftest 模块, 避免链接 DPI-C difftest。
- `vsrc/util/SimMemoryWithVirtio.sv`: 新增仿真内存包装器, 转发普通 RAM/CLINT 请求并处理 `0x10001000 virtio/simple-block MMIO`。
- `vsrc/test/simple_virtio_block_tb.sv`、`vsrc/test/ram_dpi_stubs.cpp`: 新增 simple-block MMIO 定向测试和 RAM DPI stub。
- Makefile: 新增 `test-labplus-2`、`test-labplus-3`、`test-labplus-4`、`test-labplus-pagefault`、`test-labplus-sinterrupt` 和 `test-labplus-virtio`。
- `ready-to-run/lab+/:` 补充官方 Lab+ 测试二进制和汇编反汇编文件。

6 测试结果

6.1 Lab+ atomic extension

```
make test-labplus-3
The image is ./ready-to-run/lab+/3/atomicity.bin
Core 0: HIT GOOD TRAP at pc = 0x800000dc
total guest instructions = 55
instrCnt = 55, cycleCnt = 196
```

6.2 Lab+ privfull/PMP

```
Test ecall_u [OK]
Test instr_misalign [OK]
Test instr_access_fault [OK]
Test illegal_instr [OK]
Test breakpoint [X]
Test load_misalign [OK]
Test load_fault [OK]
Test store_misalign [OK]
Test store_fault [OK]
Test timer_intr [OK]
Test software_intr [OK]
Test pmp_nr [OK]
Test pmp_nw [OK]
Test pmp_nx [OK]
Test mem_detect [OK]
    4096B user memory detected. (Interrupts: 27899)
Test m_trap [OK]
Privileged test finished.
Exit with code = 0
```

6.3 回归测试

```
Lab1 extra:
Core 0: HIT GOOD TRAP at pc = 0x8002001c
```



```
instrCnt = 32775, cycleCnt = 93022
```

Lab4:

Core 0: HIT GOOD TRAP at pc = 0x8001fff8

```
instrCnt = 32766, cycleCnt = 110574
```

Lab5:

xv6 kernel is booting

Return from init! Test passed

Lab6:

Test ecall_u [OK]

Test instr_misalign [OK]

Test load_misalign [OK]

Test store_misalign [OK]

Test timer_intr [OK]

Test software_intr [OK]

Test m_trap [OK]

Privileged test finished.

Exit with code = 0

6.4 S-mode/SRET 回归

本次新增 S-mode、委托路径、MMU page fault、PTE 权限检查、SUM/MXR、PTE A/D 硬件更新与 8-entry PMP 后重新运行：

```
make test-labplus-3
```

Core 0: HIT GOOD TRAP at pc = 0x800000dc

```
instrCnt = 55, cycleCnt = 196
```

Lab5 kernel:

Return from init! Test passed

```
TEST=sys ./build/emu --no-diff -i ./ready-to-run/lab6/lab6-test.bin
```

Privileged test finished.

Exit with code = 0

```
TEST=sys ./build/emu --no-diff -i ./ready-to-run/lab+/4/all-test-privfull.bin
```

Privileged test finished.

Exit with code = 0

6.5 MMU page fault 定向测试

```
make test-labplus-pagefault
```

instruction_page_fault [OK]

load_page_fault [OK]

store_page_fault [OK]

```
load_ok [OK]
MMU page fault directed tests passed.
```

该测试不依赖完整 CPU 或外部镜像，而是直接实例化 MMU，用简单同步 CBus 内存模型返回三层 Sv39 页表项。前三个用例分别把叶子 PTE 配成取指不可执行、load execute-only 且 MXR=0、store 不可写，确认 page_fault=1；最后一个用例使用合法可读页，确认翻译到 0x4000 且不产生 fault。

6.6 S-mode interrupt 定向测试

```
make test-labplus-sinterrupt
s_timer_interrupt [OK]
S-mode interrupt pending delegation test passed.
```

该测试直接实例化 core，手写最小指令流：M 态设置 mideleg=0x222、mie.STIE=1、stvec=0x80、mepc=0x40、mstatus.MPP=S/SIE=1，随后执行 mret。测试保持 trint=1，确认 CPU 返回 S 态后进入 S trap，并检查 scause=0x8000000000000005、sepc=0x40，从而覆盖 trint -> STIP -> S-mode timer interrupt 的转换路径。

6.7 Simple virtio block MMIO 定向测试

```
make test-labplus-virtio
virtio_magic_version [OK]
virtio_device_vendor [OK]
simple_block_capacity [OK]
simple_block_sector_size [OK]
simple_block_write_status [OK]
simple_block_read_status [OK]
simple virtio block MMIO test passed.
```

测试流程为：先读取 virtio magic/version/device/vendor 和 simple-block 容量寄存器；再通过 DPI RAM stub 在 RAM 中布置 512B pattern，向设备写 sector/mem/cmd=write；清空 RAM 后执行 cmd=read；最后逐 word 检查 RAM 中 512B 数据被恢复。

7 后续可做项

本次已经完成 xv6 主线的前四步：S-mode/trap delegation、S 态中断 pending 委托转换、MMU page fault/PTE 基础权限检查，以及一个简化同步块设备 MMIO，并补了独立 page fault、S interrupt 和 simple-block 定向测试。后续如果继续推进 xv6，需要补充：

- 完整 virtio descriptor/avail/used ring，或从 fs.img 加载初始 disk 内容。
- 更完整的 PLIC claim/complete、优先级和 enable/pending 寄存器模型。
- 更完整的 I-cache/分支预测或多级流水线，用于进一步提升 test-labplus-2 性能。

这些改动会触及 trap、CSR、MMU、取指和访存多个共享路径，风险明显高于本次 8-entry PMP、atomic、virtio 简化模型和前端空泡优化，因此本次没有继续扩大范围。

8 总结

本次 Lab+ 新增完成了 atomic extension、8-entry PMP/privfull 支持、EBREAK 断点异常、FENCE/FENCE.I 兼容, 并加入顺序取指提前、CBus fast path 和 8B 指令行缓冲等前端性能优化。继续推进 xv6 主 Track 时, 已完成 S-mode、SRET、trap delegation、S 态中断 pending 委托转换、MMU page fault/PTE 基础权限检查、SUM/MXR 权限补充、PTE A/D 位硬件更新、简化 virtio/disk MMIO, 以及独立 MMU page fault、S interrupt 和 simple-block 定向测试。AMO 实现利用现有单发访存结构, 将 AMO 指令拆成不可被其他指令插入的读-改-写序列, 并为 LR.W/SC.W 添加 reservation 状态。性能优化将 Lab1 extra 周期数从 185976 降到 93022, 将 Lab4 周期数从 208529 降到 110574, atomicity 从 372 降到 196。最终 atomic、Lab+ privileged sys-test、MMU page fault directed tests、S-mode interrupt directed test、simple virtio block MMIO test、Lab1 extra、Lab4、Lab5、Lab6 均完成回归。

9 AI 使用说明

我作为本项目的主导人, 负责实验方案制定、代码审阅、测试验证和最终提交确认。AI (Codex) 用于辅助阅读 Lab+ 要求、分析官方 atomicity.S 和 all-test-privfull.S、实现 AMO/PMP/性能优化、定位 difftest atomic event 与 trap 重定向时序问题, 并整理实验报告。最终代码和报告由我本人审阅确认。